Structures de haut niveau en assembleur

Les variables

- Les variables et tableaux sont alloués dans le segment de données
- Chaque variable et tableau est déclaré avec un label donné et une taille et pour les tableaux la taille maximale du tableau
- Les tailles possibles des variables sont:
  - DB: octet (8 bits)
  - DW: mot (16 bits)
  - DD: double (32 bits)

Les variables

- Exemple de déclaration:

```
data SEGMENT
A        db 1
B        db -1
C        DW  342
Tab      DD  100 dup(0)
Tab2     DB  12 dup(?)
data ENDS
```

- Les registres généraux (AX, BX, CX, DX) peuvent être utilisés pour stocker des variables temporaires

Les expressions arithmétiques et logiques

- La traduction en assembleur d'une expression arithmétique consiste à la décomposer en opérations ayant un ou deux opérandes source et un opérande cible

- Exemple:

Langage C

```
z = a + b - c
```

Assembleur

```
debut :   MOV AX,a
          ADD AX,b
          SUB AX,c
          MOV z,AX
```

Les expressions arithmétiques et logiques

- Une séquence d'instructions se traduit instruction par instruction de manière séquentielle
- Exemple:

Langage C

```
x = a - b + c;
y = d - x;
z = x + y
```

Assembleur

```
; instruction x = a - b
+ c
debut:    MOV AX,a
          SUB AX,b
          ADD AX,c
          MOV x,AX
; instruction y = d - x
          MOV BX,d
          SUB BX,AX
          MOV y,BX
; instruction z = x + y
          ADD BX,AX
          MOV z,BX
```

Les tests conditionnels

- La structure générale d'un test est la suivante

Langage C

```
if (condition)
{

    /* … action-then */

}
else
{

    /* … action-else */

}
```

Assembleur

```
          calcul de la condition
          Jcc   etiquette_sinon
          ; intructions de action-th
          action-then
          ...
          JMP etiquette_fin_si
etiquette_sinon :
          ; intructions de action-si
          action-else
          ...
etiquette_fin_si : ;
          suite du programme
```

Le ca

- La condition est une condition booléenne, c'est à dire une expression dont la valeur est vrai ou faux. Il peut s'agir d'une variable booléenne ou d'une condition

Les tests conditionnels

- Le calcul de la condition revient à évaluer une expression
- Ce calcul positionne certains bits du registre d'état dont l'un sera testé par une instruction de saut conditionnel (instruction Jcc à choisir en fonction du but à tester) pour décider laquelle des actions (action then ou action else) doit être exécutée
- Une fois la structure de test conditionnel traduite il reste à traduire les deux blocs d'instructions formant les actions

Les conditions simples

- Les conditions simples ont la forme suivante:
  - $\text{expression1} = \text{expression2}$
  - $\text{expression1} \neq \text{expression2}$
  - $\text{expression1} < \text{expression2}$
  - $\text{expression1} > \text{expression2}$
  - $\text{expression1} \leq \text{expression2}$
  - $\text{expression1} \geq \text{expression2}$
- Expression1 et expression2 sont des expressions arithmétiques ou de simples variables
- Ces expressions sont évaluées et on compare leur résultat à l'aide de l'instruction CMP

Les conditions simples

Langage C

```
if (e1 == e2)
{

 /* … action-then */

}
else
{

 /* … action-else */

}
```

Assembleur

```
          CMP e1,e2
          JNE   etiquette_sinon
          ; intructions de action-then
          action-then
          ...
          JMP etiquette_fin_si
etiquette_sinon :
          ; intructions de action-sinon
          action-else
          ...
etiquette_fin_si : ;
          suite du programme
```

Les conditions simples

- Pour une condition donnée on utilisera une séquence de deux instructions: une instruction CMP de comparaison suivie d'une instruction de saut conditionnel Jcc

Les conditions simples

- Exemple avec une expression

Langage C

```
if (a+b == 10)
{

 /* … action-then */

}
else
{

 /* … action-else */

}
```

Assembleur

```
          mov AX,a
          add AX,b
          CMP AX,10
          JNE   etiquette_sinon
          ; intructions de action-then
          action-then
          ...
          JMP etiquette_fin_si
etiquette_sinon :
          ; intructions de action-sinon
          action-else
          ...
etiquette_fin_si : ;
          suite du programme
```

Les conditions composées

- Une condition composée est constituée par des conditions simples reliées par des opérateurs ET, OU, ou la négation d'une condition:

  - condition1 ET condition2
  - condition1 OU condition2
  - NON condition

- La condition composée est vraie si les deux conditions le sont

Les conditions composées

- Condition 1 et condition 2: la condition composée est vraie si les deux conditions le sont

Langage C

```
if (cond1 && cond2)
{

 /* … action-then */

}
else
{

 /* … action-else */

}
```

Assembleur

```
 THEN     calcul condition1
          Jcc  etiquette_sinon ; condition1 faussse
          calcul condition2
          Jcc  etiquette_sinon ; condition2 faussse
          ; intructions de action-then
          action-then
          ...
          JMP etiquette_fin_si
etiquette_sinon :
          ; intructions de action-sinon
          action-else
          ...
etiquette_fin_si : ;
          suite du programme
```

Les conditions composées

- Condition 1 ou condition 2: la condition composée est vraie si l'une des deux conditions l'est

Langage C

```
if (cond1 || cond2)
{

 /* … action-then */

}
else
{

 /* … action-else */

}
```

Assembleur

```
          calcul condition1
          Jcc  etiquette_calc_cond_2 ; condition1 faussse
          JMP etiquette_then
etiquette_calc_cond_2 :
          ; calcul de la condition2
          calcul condition2
          Jcc  etiquette_sinon ; condition2 faussse
etiquette_then :
          ; intructions de action-then
          ; action-then
          ...
          JMP etiquette_fin_si
etiquette_sinon :
          ; intructions de action-sinon
          action-else
          ...
etiquette_fin_si : ;
          suite du programme
```

Les conditions composées

- Négation d'une condition: peut être vue comme une condition simple en considérant son opposé

Langage C

```
if (a+b == 10 || c+d==2)
{

 /* … action-then */

}
else
{

 /* … action-else */

}
```

Assembleur

```
          MOV AX,A
          ADD AX,B
          CMP AX,10
          JNE  etiquette_sinon ; condition1 faussse
          MOV BX,C
          ADD BX,D
          CMP BX,2
          JNE  etiquette_sinon ; condition2 faussse
          ; intructions de action-then
          action-then
          ...
          JMP etiquette_fin_si
etiquette_sinon :
          ; intructions de action-sinon
          action-else
          ...
etiquette_fin_si : ;
          suite du programme
```

Les boucles

- Les boucles permettent de répéter une suite quelconque d'instructions
- Le nombre d'itérations peut être déterministe (fixé à l'avance) ou conditionnel (dépendant d'une expression booléenne)
- Dans le cas où le nombre d'itération dépend d'une condition, l'évaluation de la validité de la condition peut s'effectuer avant ou après l'exécution de la séquence d'instructions à répéter

La structure TANT QUE (while)

- Elle consiste à exécuter une séquence d'instructions tant qu'une condition booléenne est vérifiée (valeur vraie)
- Le groupe d'instructions exécutée si la condition est vrai s'appelle le corps de la boucle
- Ce type de boucle peut se traduire par un test reposant sur la valeur de la condition
- En fonction du résultat du test, cette évaluation sera suivie soit par l'exécution du corps de la boucle, soit par un saut hors de la boucle à l'instruction qui la suit
- Si le corps de la boucle est exécuté la condition doit ensuite être à nouveau calculée puis testée afin de déterminer si l'exécution de la boucle de se poursuivre ou non

La structure TANT QUE (while)

Langage C

```
while (condition) {
    /*… action_boucle */
}
```

Assembleur

```
debut_boucle : ;
          calcul cond
          Jcc  fin_boucle ; condition1 faussse
          ..
          ; instructions action_boucle
          ...
          JMP debut_boucle
fin_boucle : ;
          suite du programme
```

La structure REPÉTER

- Comme la structure TANT QUE, elle dépend d'une condition
- A la différence de la structure TANT QUE, la condition est évaluée après exécution du corps de la boucle
- Quelle que soit la valeur de la condition, le corps de la boucle sera exécuté au moins une fois
- Elle correspond à la structure "do while" en langage C

La structure RÉPÉTER

Langage C

```
do {
    /*… action_boucle */

} while(condition);
```

Assembleur

```
debut_boucle:
          …

          ; instructions action_boucle
          …


          ; calcul cond
          Jcc debut_boucle ; si condition vraie

          ; suite du programme
```

La structure POUR (for)

- Ce type de boucle doit être utilisé lorsque l'on connaît explicitement le nombre de fois que le corps de la boucle doit être exécuté

Langage C

```
for(int i=valeur_init;i<=valeur_finale;i++)
{
          /* … action_boucle */
}
```

Assembleur

```
          MOV CX, valeur_init
debut_boucle:
          CMP CX,valeur_finale
          JG fin_boucle
          …
          ; instructions action_boucle
          …
          INC CX
          JMP debut_boucle
fin_boucle:
          ; suite du programme
```

La structure POUR (for)

- L'instruction loop permet d'instancier une structure POUR d'une valeur initiale quelconque N jusqu'à la valeur finale 0, par pas de -1
- La valeur initiale doit être stockée dans le registre CX
- L'instruction loop gère automatiquement la valeur de CX de la manière suivante:
  - la valeur de CX est décrémentée
  - si elle est différente de 0, l'instruction loop branche à l'instruction repérée par l'étiquette donnée en opérande de manière à revenir au début du corps de la boucle
- Il ne faut donc pas modifier la valeur de CX dans le corps de la boucle

La structure POUR (for)

- Exemple avec l'instruction loop

Langage C

```
for(int i=valeur_finale;i<=valeur_init;i--)
{

          … action_boucle

}
```

Assembleur

```
          MOV CX, valeur_finale
debut_boucle:

          …
          ; instructions action_boucle
          …
          loop debut_boucle
fin_boucle:
          ; suite du programme
```

Les boucles (exemples)

- Somme des N éléments d'un tableau TAB

Langage C

```
Somme = 0;

for(i=0;i<N;i++)
    Somme += TAB[i];
```

Assembleur

```
            MOV AX,0 ;
            MOV CX, 1
            MOV SI,0; indice TAB
debut_boucle : ;
           CMP CX, N
           JG fin_boucle ;
           ADD AX,TAB[SI]
           INC SI
           INC CX
           JMP debut_boucle
fin_boucle : ;
           suite du programme
```

Les boucles (exemples)

- Recherche d'un élément dans un tableau et détermination de la position de cet élément dans le tableau

Langage C

```
i = 0;
index = -1;

while(i < N && TAB[i] != Element)
 i++;

if(i < N)
    index = i;
```

Assembleur

```
            MOV BX,element
            MOV AX,0 ;
            MOV SI,0; indice TAB
debut_boucle : ;
           CMP SI, N - 1
           JG fin_boucle ;
           CMP BX,TAB[SI];
           JE fin_boucle
           INC SI
           INC CX
           JMP debut_boucle
fin_boucle :
           CMP SI, N - 1
           JG fin_si
           MOV index,SI
fin_si      : ; suite programme
```